

Annex X — WR Guard™ and Capability-Governed Information Flow for Hybrid AI

BEAP™ / WR Desk™ / Optirando™ Canonical Specification — standalone Annex. Version 1.2 (see §X.17). Document status: Normative unless explicitly marked as informative.

Applies to: the internal information-flow surface of the Optirando™ system — the bidirectional flows between local devices, internal components, field devices, sandbox environments, local applications and services, local AI models, the WR Graph™, and external cloud services and APIs; the governance fabric under which every such flow is governed (Public Handshakes per Annex IX; Internal Handshakes and External Service Admissions per this Annex); capability-scoped access to the WR Graph™ and other protected data; the WR Expert™ file class and the WR Guard™ as the general local supervision component over the stack's data streams and workflows, with real-time egress analysis of direct operator input as its first duty area; the graded egress measures and their deterministic enforcement; the Enforcement Mode discipline for local analysis components; and the cryptographic evidence discipline for every boundary crossing and every analysis finding.

Trademark notice. Optirando™, WR Desk™, WR Graph™, BEAP™, PoAE™, PoAC™, WRVault™, Fraud Watchdog™, WR Code®, **WR Expert™**, and **WR Guard™** are used throughout as product and protocol designations of the Optirando™ system. The names **WR Expert™**, **WR Guard™**, **Internal Handshake**, and **External Service Admission** are registered within the Canon to prevent divergent usage.

X.0 Status, Scope, and Relationship to the Canon

This Annex specifies the trust discipline of the system's interior: the principle that no component — however local, however internal — obtains access to data or functions by membership alone, and the mechanism set through which every information flow into and out of the system is cryptographically negotiated, capability-bounded, analyzed where policy demands it, and evidenced without exception.

This Annex does not redefine wire formats, capsule serialization, cryptographic primitives, transport, handshake profile mechanics, or the artifact and manifest machinery. It depends on, reuses, and does not weaken:

- **Annex II** — the non-pre-satisfaction rule (§II.2.5): information never authorizes; graceful degradation (§II.2.8); WR Graph™ structure and routing;
- **Annex III** — the PoAC™ capability-acquisition discipline governing skills, recordings, adapters, and prompt profiles; "propose, never silently apply";
- **Annex VII** — handshake profile machinery, capability declaration, the grant model, revocation, publisher and endpoint attestation;
- **Annex IX** — the Public Handshake and its initiation layer (IX.3); the consequence-boundary model (IX.8), including the completeness rule that reads and transmissions are consequences (IX.8.4(3)); Execution Capabilities (IX.8.5); the Closed Model Context (IX.8.6); Plausibility Verification and the ratchet discipline (IX.8.7); Effect Binding (IX.8.8); the Consent Mechanism Classes C0–C4 (IX.18); the PoAE™ record, Execution Receipt, and

Intent Hash (IX.19); the Verification Tiers L/C/X (IX.20); the Transaction Envelope (IX.21); and the enforcement-substrate mapping (IX.22).

Nothing in this Annex modifies, relaxes, or re-defines any rule of Annexes II–IX. Where this Annex restates an existing rule, the restatement is a surface-specific view of the identical rule, and the existing section remains authoritative. In particular, this Annex introduces **no new enforcement layer**: crossings of a trust boundary are consequences in the sense of IX.8.4(3) and are governed by the existing consequence boundary applied to a further surface. The core philosophy of the Canon applies without exception: **the system prepares, the human authorizes, the system records cryptographic proof**.

X.0.1 Claims Discipline (Normative)

The claims discipline of Annex IX §IX.0.1 applies to this Annex without exception. This Annex describes an architecture designed to govern and evidence internal and external information flows as a long-term objective. It SHALL NOT be represented — in specification text, product text, or derived material — as already constituting a certified or formally verified system, and SHALL NOT be represented as guaranteeing that protected data can never leak. Statements are of two kinds and are marked accordingly: properties that hold **by construction** of the specified mechanism, and **objectives** requiring implementation, testing, adversarial analysis, and audit before they may be claimed (X.13). This distinction is load-bearing for a beta-stage system and SHALL be preserved in all derived material.

X.1 Motivation, Problem Statement, and Threat Context (Informative)

X.1.1 The Problem — Trust by Address

The problem this Annex exists to solve can be stated in one sentence: **an operator should be able to work with local components, internal services, and external AI surfaces without any of them — and without the operator's own inadvertence — carrying protected information across a trust boundary that no explicit, cryptographically verifiable agreement covers**.

Conventional architectures fail this in two symmetric ways. First, they grant **trust by address**: what runs on the same machine, inside the same network perimeter, or under the same organizational account is treated as entitled — local processes read local data, internal services query internal stores, and a sandbox inherits the environment of its host, because locality is silently equated with trustworthiness. Decades of lateral-movement, privilege-escalation, and supply-chain incidents demonstrate what that equation is worth. Second, they leave the most consequential modern egress path — the operator typing into a cloud AI prompt — governed by nothing but attention. The employee who pastes a customer table, an internal roadmap, or credentials into a chat window commits no attack; the architecture simply provided no boundary at the point where the boundary mattered, and no record afterwards that anything crossed at all.

Zero-trust networking addresses the first failure at the network and identity layer — authenticate every request, trust no perimeter — but stops short of the object: it authenticates *who is calling*, not *what may flow, to whom, for what purpose, under which transformations, with what proof*. Data-loss-prevention tooling addresses the second failure with pattern matching at gateways — but as a

detached filter: heuristic, unaccountable to a consent record, blind to semantics beyond its patterns, and typically producing logs that prove nothing cryptographically. Neither supplies a contract.

X.1.2 What Must Be Prevented — and What Prevents It (Informative; mechanisms normative where cross-referenced)

Threat	Why it is addressed here
Lateral access by co-location — a local process, sandbox, or internal service reading data because it is "inside"	Trust is not implied by locality (X.4): every component holds exactly its handshake-negotiated capabilities, and a sandbox inherits nothing; access without a capability is not a denied operation but an unrepresentable one; <i>by construction</i> .
Silent, unbounded data flows between components	Every flow direction of every relationship is separately declared and capability-covered under a Public Handshake, an Internal Handshake, or an External Service Admission (X.3); an undeclared flow has no authorized path; <i>by construction</i> .
Unscoped harvesting of the WR Graph™ by internal or external requesters	Graph access is authorized per operation by a capability declaring regions, operation, recipient, purpose, protections, validity, and revocation (X.5); <i>by construction</i> .
Inadvertent disclosure of protected information in cloud AI prompts	Deterministic detectors for registered material apply their policy-mapped measure unconditionally; the WR Guard™ classifies semantically in real time during typing; measures range from marking to hard block; releases require graded consent over the marked presentation (X.7–X.8); deterministic layer <i>by construction</i> , semantic layer an <i>objective</i> (X.13).
A deceived, wrong, or compromised analysis model releasing data	No finding, absence of a finding, or model output can release, soften, or authorize anything in any Enforcement Mode; the model holds no capability material and the ratchet discipline is mode-independent (X.8, X.9; IX.8.5, IX.8.7); <i>by construction</i> .
Unevidenced boundary crossings — data leaves, and nothing proves what, when, under which authority	Every crossing of a declared trust boundary, and every analysis finding, is sealed into the append-only, hash-chained evidence log and verifiable at the configured tiers (X.10; IX.19–IX.20); <i>by construction</i> — noting that evidence proves, it does not prevent (X.0.1).
Profiling of the operator through the analysis layer itself	WR Guard™ analysis runs locally; neither findings nor analyzed content are transmitted off-device for this purpose; higher verification tiers carry hash commitments only (X.7, X.10; IX.11, IX.20); <i>by construction</i> .
Anomalous internal activity — a compromised or misbehaving component operating within its formal capabilities	The WR Guard™ supervises data streams and workflows against declared expectations and the historical corridor; findings add friction up to suspension pending release, per Enforcement Mode, while the deterministic evidence chain proves what occurred (X.7.7, X.10); heuristic layer an <i>objective</i> (X.13), the evidence <i>by construction</i> .
Policy circumvention by a counterparty — a publisher or external service demanding weaker analysis, lower consent, or a lower evidence tier	Meet-or-exceed applies to Enforcement Modes, Consent Mechanism Classes, and verification tiers alike: a counterparty can never lower what the operator's policy demands (X.8, X.9; IX.18.3, IX.20); <i>by construction</i> .

X.1.3 The Trust Inversion, Interior Edition (Informative)

Annex IX inverts trust for the unknown publisher: safety derives from cryptographic agreement, not from the counterparty's virtue. This Annex performs the identical inversion for the *known* component — and the discipline is needed precisely because familiarity breeds implicit grants. The field device commissioned last year, the sandbox spawned a millisecond ago, the model fine-tuned in-house, and the cloud API under contract are all treated identically in discipline: as counterparties whose every data relationship is explicitly declared and bounded — negotiated where the counterparty speaks WR, unilaterally admitted where it does not (X.3.3) — capability-bounded per direction, and evidenced per crossing. The fix is structural, not evaluative — which is why it holds equally for the compromised internal component and for the merely misconfigured one.

X.2 Definitions

Terms defined in Annexes II–IX apply as defined there; in particular Public Handshake, Handshake Contract, Scope Envelope, Execution Capability, Consent Mechanism Class, Intent Hash, Execution Receipt, Verification Tier, WR Trusted Execution Domain. This Annex adds:

Trust Domain. A deployment-declared boundary of governance: the set of devices, components, and stores operating under one administrative authority and one signed local policy. Trust domains nest and federate; crossing any declared trust-domain boundary is a boundary crossing in the sense of X.10 regardless of physical locality.

Internal Handshake. The canonical designation for a handshake formed between components of the system — device ↔ device, component ↔ component, host ↔ sandbox, host ↔ field device, client ↔ local service, client ↔ external API — under the Annex VII profile machinery, as distinct from the Public Handshake of Annex IX (operator ↔ publisher). An Internal Handshake yields a Handshake Contract with **per-direction capability sets** (X.3.2) and is subject to the identical grant, revocation, and evidence discipline. The name is registered to prevent divergent usage; profile-level wire semantics remain defined by Annex VII.

External Service Admission. The unilateral, deployment-side governance artifact for a non-WR external service (a cloud AI API, an external RAG or search endpoint, any third-party service that forms no WR handshake): a signed, versioned declaration — admitted as a policy or consent event under the artifact discipline of Annex III — of the service's endpoints and identity anchors, the permitted operation classes, the per-direction data-class ceilings, mandatory Egress Measures, the required Invocation Assurance Class per operation class, and the evidence requirements. An admission is not a handshake: the counterparty forms nothing, attests nothing, and holds no WR identity; all governance is client-side (X.3.3). The name is registered to prevent divergent usage.

External Invocation. A single governed operation against an admitted external service — a completion request, a retrieval or search query, an embedding call. Each invocation is authorized individually against the admission's declared ceilings, carries both directions (query/prompt egress and response/result ingress), and is evidenced as a Boundary Event Record (X.10).

Invocation Assurance Class. The declared assurance grading of an External Invocation's operation class: the minimum Consent Mechanism Class (IX.18) or admitted C0 policy rule authorizing it, the mandatory Egress Measures and transformations, the Enforcement Mode floors for the analyzing components, and the minimum Verification Tier of its evidence. Assurance classes are declared per

operation class and per data class in the admission and the signed policy; a service SHALL NOT be able to lower any of them (the meet-or-exceed discipline of IX.18.3 applied to invocations).

WR Expert™ File. A governed knowledge artifact of the Optirando™ system: a versioned, administrator-signed, SHA-256-identified file (for example, a structured Markdown document) that parameterizes a local model for a defined expert role. WR Expert™ files are the WR system's skill-file class — carrying domain knowledge, classification rules, exclusion and release rules, and semantic scope definitions for the consuming component — and are governed as capability artifacts under the PoAC™ discipline of Annex III: hash-pinned, monotonically versioned, admitted only through a deliberate policy or consent event, never silently activated or replaced. The confidentiality profile consumed by the WR Guard™ (X.7) is one instantiation of this class; the class itself is general. The complete class specification is an open item (X.15).

WR Guard™. The designation of the WR Desk™ security supervision, routing, and orchestration architecture (X.7). WR Guard™ is not itself a LoRA-class model or adapter. It provides the governed framework within which specialized local security adapters, analysis components, WR Expert™ files, tools, and deterministic enforcement mechanisms are coordinated across the stack's data streams and workflows. Its duty areas at this version (X.7.1) include real-time analysis of operator input before it crosses an external trust boundary, supervision of ingress and egress information flows between components and across declared boundaries, and supervision of workflow and automation execution.

Specialized security functions are provided by domain-specific LoRA-class adapters operating within WR Guard™, including **Fraud Watchdog™** for fraud, phishing, and social-engineering analysis, **Ingress Watchdog™** for ingress, egress, untrusted-input, and boundary-risk analysis, and **Process Watchdog™** for process behavior, runtime anomalies, and workflow-related risk detection. These adapters may be activated explicitly as selectable modes or dynamically hot-swapped by the WR Guard™ Smart Routing Engine where context, policy, or a high-priority risk condition requires specialized analysis. Adapter availability does not imply automatic activation; many adapters remain user- or workflow-selectable and are routed automatically only where the applicable policy and routing conditions permit it.

WR Guard™ findings produced by probabilistic analysis are heuristic in origin; their consequences remain governed by the policy-declared Enforcement Mode (X.9). The architecture therefore preserves the internal separation defined in X.7.8 between a probabilistic **analysis and classification stage**, which may employ one or more specialized adapters, and a deterministic **enforcement stage**, which verifies the cryptographically signed policy and applies and logs the corresponding mapped decision at the Guard boundary. Neither an adapter, the Smart Routing Engine, nor any probabilistic model possesses release, execution, or policy authority. Authority derives exclusively from the signed policy and the deterministic enforcement path.

First-Party Adapter Integrity and Advance Analysis (normative)

WR Guard adapters are **first-party artifacts issued, versioned, and signed by Optirando**. The runtime hot-swaps adapters according to the current operational context, but admits only adapters whose signature and content hash verify against the Optirando trust anchor. A forged or modified adapter cannot enter the Guard through any channel.

Policy MAY declare a **minimum admissible version per adapter class**; no swap below the declared minimum is permitted (rollback protection against downgrade to a correctly signed but superseded adapter).

Every verdict emitted by WR Guard records the **version of the adapter that produced it**, making each verdict reproducible against a known, signed artifact.

Guard analysis is not confined to the interactive path. During idle phases, the same governed pipeline MAY perform **advance analysis** of already-depackaged inbound content — including e-mail under the Annex IX depackaging and channel-provenance discipline: identical adapter set, identical policy, verdicts recorded with the producing adapter version as provenance records. When the user subsequently opens the item, only the previously verified result is surfaced, eliminating interactive latency. Advance analysis changes evaluation **timing** only; it confers no additional authority and never crosses a trust boundary that the interactive path would not cross.

Deterministic Confidentiality Detector. A non-model detection rule of the egress pipeline (X.7.3): exact terms and identifiers declared in a WR Expert™ file, tagged data classes, document fingerprints of registered confidential material, and content of WR Graph™ regions marked confidential. A detector match is a fact, not a finding, and knows no Enforcement Mode.

Egress Measure. One of the graded, policy-mapped protective responses of X.8 applied by the deterministic policy layer to classified content. Each measure is a declared processing path; selection among applicable measures follows the router discipline of IX.16.4.

Distribution. The Egress Measure that splits information across several models, services, or processing steps such that no single external recipient receives the complete sensitive context (X.8.2).

Enforcement Mode. The policy-declared operating mode — **advisory** or **enforced** — governing the consequence of findings produced by a local analysis component, declared in the signed local policy per component and per finding class (X.9). The mode governs consequence, never epistemic status.

Boundary Event Record. The sealed evidence entry of X.10: the record of a boundary crossing (X.10.2) or of an analysis finding (X.10.3), appended to the per-domain hash chain under the Execution Receipt discipline of IX.19 and verifiable at the configured Verification Tiers (IX.20).

X.3 The Governance Fabric (Normative)

X.3.1 One Fabric, Three Governance Classes

Every data relationship of the system — inward or outward, internal or external — SHALL be governed under exactly one of three governance classes:

- the **Public Handshake** of Annex IX, where the counterparty is a publisher entered through the initiation layer (WR Code® scan, manual entry, Assisted Code Capture);
- the **Internal Handshake** of this Annex, where the counterparty is a WR-speaking component: another device of the deployment, a sandbox or depackaging context, a field device, a local application or background service, or a local AI model runtime; and

- the **External Service Admission** of this Annex (X.3.3), where the counterparty is a non-WR external service — a cloud AI API, an external RAG or search endpoint — that forms no handshake: governance is unilateral, declared client-side, and enforced per invocation.

No fourth governance class exists. A component without a formed handshake, and an external service without an admission, holds no capability; a flow outside every handshake and every admission has no authorized path (X.4). Internal Handshake formation follows the Annex VII profile machinery — identity binding, capability declaration, grant model, unilateral revocation — with formation authority resting in deployment policy (components do not self-admit): admission of a component and of its capability sets is a policy or consent event of the administrative authority, hash-pinned and evidenced, under the same "propose, never silently apply" discipline that governs every capability acquisition (Annex III).

X.3.2 Bidirectionality — Per-Direction Capability Sets

A handshake governs a *relationship*, and a relationship has two directions. The Handshake Contract of every Internal Handshake, and of every Public Handshake where the deployment's data participates, SHALL declare its capability sets **per direction**:

- the **ingress set** — what may enter from the counterparty: admitted data classes, sources, and formats, under the context-admission discipline of IX.8.6 where a model consumes them (typed, bounded, consent-admitted; a non-admitted source is unrepresentable as context);
- the **egress set** — what may leave toward the counterparty: data classes, graph scopes, purposes, transformations (X.8), recipients, and validity.

Neither direction implies the other: a component authorized to deliver telemetry inward is not thereby authorized to read anything outward, and a service authorized to receive abstracted summaries is not thereby authorized to submit content into the trust domain. An undeclared direction is an empty capability set, and a flow in that direction is a deviation (IX.6.5). An External Service Admission declares the identical per-direction structure (X.3.3); the directional grammar is uniform across all three governance classes. The bidirectional declaration is the interior form of the Scope Envelope discipline (IX.5.1): consent and policy attach to the declared directional envelopes and to specific operations — never to content, so that the consent record remains permanently accurate while content moves within the envelopes.

The system thereby controls, under one uniform cryptographic discipline, the full diversity of its data flows: device → graph reads, graph → device disclosures, host → sandbox provisioning, sandbox → host result return, field-device synchronization in both directions, operator → cloud prompts, cloud → operator responses, API calls and their returns, and every mixed path composed of these. Uniformity is the point: there is one grammar of authorization, whatever the flow.

X.3.3 The External Service Boundary — Admission, Not Handshake

External cloud services and APIs — model providers, external RAG and search endpoints above all — are a governance class of their own. Strictly, their consumption is **not a handshake**: the counterparty implements no WR protocol, forms nothing, attests nothing, and holds no WR identity; there is no mutual capability negotiation and no counterparty-side enforcement to rely on. Treating such a service as a handshake party would misstate the trust model. Instead:

1. **Admission.** An external service is usable only under an External Service Admission (X.2): the deployment-side, signed, versioned declaration of endpoints and identity anchors (TLS/WebPKI; credential custody in the client core per IX.10), permitted operation classes, per-direction data-class ceilings, mandatory Egress Measures, required Invocation Assurance Classes, and evidence requirements. Admission, and every change to it, is a policy or consent event under the artifact discipline; a non-admitted service is unreachable as an authorized target.
2. **Per-invocation authorization, graded by assurance class.** Authorization is fine-granular: each External Invocation is authorized individually against the admission, at the Invocation Assurance Class declared for its operation class and the data classes involved — from C0 policy-autonomous invocations for low classes (e.g. an abstracted retrieval query under an admitted rule) through interactive consent up to hardware-backed classes for the most sensitive (IX.18; meet-or-exceed unchanged). Higher data classes MAY additionally mandate transformations — masking, abstraction, Distribution, recipient-bound encryption (X.8) — as a precondition of the invocation, not merely as an option.
3. **Both directions, per invocation.** The egress direction — the prompt, the RAG query, the search string — is disclosure-bearing content crossing the boundary: WR Guard™ duty area 1 and the Deterministic Confidentiality Detectors apply (X.7), and the query itself is recognized as disclosure (a retrieval query can leak as much as a prompt). The ingress direction — the completion, the retrieved passages, the search results — is external free text: it is admitted under the context-admission discipline (IX.8.6) as a declared free-text source, consent-covered, with the residual influence bounded exactly as there stated — consumable as information, incapable of instruction, never widening any space (Annex II §II.2.5); admission validation applies before any downstream consumption.
4. **Evidence.** Every invocation is a Boundary Event Record (X.10): admission reference and version, operation class, satisfied assurance class, data-class commitments of both directions, applied transformations, and the consent or policy reference. The Cloud Abstraction Boundary of the Canon is unchanged and sits above this discipline where it applies (cloud reasoning at taxonomy level, never on operator data); this Annex adds the per-admission declaration, the per-invocation authorization, and the per-crossing proof beneath it.

The distinction pays where it matters: a handshake distributes trust across two attested endpoints; an admission concentrates the entire governance on the client side — which is the honest description of consuming a service that guarantees nothing in WR terms. Should an external provider later implement WR — attested identity, declared capabilities, counterparty-side Finalizers — the relationship upgrades to an ordinary handshake class through an ordinary formation event; nothing accumulated under the admission pre-satisfies that formation (Annex II §II.2.5).

X.4 Locality-Independent Trust (Normative)

Generating principle: trust is not implied by locality. Membership in the same platform, deployment, device, network, or trust domain SHALL NOT by itself ground any access to data or functions. Internal devices, sandbox processes, field devices, background services, local applications, and local AI models are subject to the identical discipline as external cloud services: each receives exclusively the capabilities explicitly negotiated through an authenticated handshake

(X.3), and every information flow across a trust boundary — inbound or outbound — is authorized per operation by a cryptographically verifiable capability.

The handshake confirms identities and enables the cryptographically verifiable agreement of scope, purpose, permitted data classes, processing steps, recipients, validity period, and further policies. It does not replace object- or purpose-bound authorization: a formed handshake yields the agreed capability sets and nothing beyond them (Annex II §II.2.5 applied to internal components — a relationship informs; it never authorizes).

Within the controlled rendering and execution pipeline, the architecture is designed so that, absent a valid capability, no authorized path exists to read, disclose, transmit, or modify protected data. Consistent with X.0.1, this is a statement about the specified mechanism; its complete effectiveness presupposes that no uncontrolled side paths, alternative network accesses, privileged processes, or compromised components bypass the controlled pipeline (X.13).

Two corollaries, stated for the avoidance of doubt:

1. **A sandbox inherits nothing.** A sandbox or depackaging context receives no host data, no host keys, no WR Graph™ access, and no standing capability by virtue of running on the host. Whatever it processes was passed to it under an explicit capability of its Internal Handshake; whatever it produces re-crosses the boundary only under one (blob-inertness of the BEAP™ depackaging model unchanged).
2. **The local model is an untrusted probabilistic component.** It may analyze, classify, summarize, select, and generate within its admitted context (IX.8.6), but it holds no capability material (IX.8.5, IX.10(2)) and cannot authorize any ingress or egress. This is the router discipline of IX.16.4 restated for the interior: the AI routes; it never paves — and it never opens a door.

X.5 Capability-Scoped Access to the WR Graph™ and Protected Data (Normative)

The WR Graph™ serves as the authoritative knowledge, context, relationship, and capability store of the system. Neither external cloud services nor internal field devices, sandbox processes, or other components obtain automatic or complete access to it. Access is graded by **requester class** (below), and for every non-resident requester it SHALL be authorized per relevant operation by a cryptographically verifiable capability — carried under the requester's Internal or Public Handshake, or its External Service Admission — which defines, at minimum:

1. the affected graph regions, nodes, attributes, branches, or information classes;
2. the permitted operation;
3. the recipient to whom information may be disclosed;
4. the purpose of processing;
5. the protective measures to be applied (including, where applicable, the Egress Measures of X.8);
6. the validity period of the authorization; and
7. the conditions under which it is revoked or refused.

This is the Scope Envelope discipline (IX.5.1) and the context-admission discipline (IX.8.6) applied to the graph surface. Data retrieved under a capability informs processing and never authorizes

further disclosure (Annex II §II.2.5); onward disclosure is a new egress under its own capability. A request outside the declared scope is not a denied operation — it is unrepresentable as an authorized one, and the attempt is a deviation (IX.6.5) with a Boundary Event Record (X.10).

Requester access classes. The capability requirement is graded by requester class, not by content marking alone:

1. **Resident access — local models.** A local model runtime operating within the host's trusted environment MAY access the WR Graph™ as working context — the graph is its local memory — subject to the admitted-context discipline (IX.8.6) and to any exclusions the admitted WR Expert™ files declare for marked regions. Resident access requires no per-operation cryptographic approval ceremony and is recorded in the local evidence log. This is not trust by locality (X.4): resident access is an explicitly declared class of the signed policy; it conveys information and never disclosure authority — nothing read into local inference crosses any boundary except under the egress discipline of this Annex — and it is confined to runtimes inside the trust domain under the isolation invariants of IX.10, holding no capability material.
2. **Approved internal access — field devices and dedicated sandboxes.** Non-resident internal device classes — field devices, dedicated sandboxes, and comparable components — SHALL always require cryptographic approval: a per-operation capability under their Internal Handshake, for every access, regardless of content classification. The requirement attaches to the requester class; content marking (confidential regions) adds restrictions on top of it and never substitutes for it.
3. **External access.** External sources and services SHALL always require cryptographic approval; disclosure toward them additionally passes the egress discipline (X.8) and, for non-WR services, the admission ceilings and Invocation Assurance Classes of X.3.3.

Every graph access is logged: classes 2 and 3 as Boundary Event Records (X.10), resident access in the local evidence log per IX.11.

The concrete internal graph structure is deliberately not fixed by this Annex. Normative is the general principle of a controlled, cryptographically authorized retrieval and disclosure path, uniformly applied to internal and external requesters; structural addressing belongs to the WR Graph™'s own specification (Annex II), and the capability field grammar is an open item (X.15).

X.6 The WR Expert™ File Class (Normative)

The deployment's administrative authority (the local policy owner) authors and signs WR Expert™ files: versioned knowledge artifacts that parameterize local models for defined expert roles (X.2). A confidentiality-profile WR Expert™ file describes precisely which information the device or the respective trust domain must not disclose, and MAY declare, among others: location and site information; personal data and other PII; company-internal information and trade secrets; credentials, keys, and tokens; customer data; confidential project information; regulatorily protected information; topic areas specially marked in the WR Graph™; semantically related terms and topic fields; context-dependent risk classes; and organization-specific exclusion and release rules.

WR Expert™ files are admitted, versioned, and replaced exclusively under the PoAC™ artifact discipline: admission and every version transition is a deliberate policy event, hash-pinned and recorded in evidence; silent replacement does not exist as a mechanism. A WR Expert™ file binds

the risk-class vocabulary against which the signed egress policy (X.8.3) declares its measure mapping; file and policy version together define the rule state attributable to every classification (X.7.5).

X.7 The WR Guard™ (Normative)

X.7.1 Position and Duty Areas

The WR Guard™ is the deployment's general supervision component: it watches over the activities of the stack — its data streams and its workflows — as the heuristic layer above the deterministic mechanisms of the corpus, which remain authoritative in every case. Its duty areas at this version are:

1. **Operator-input egress analysis** (X.7.2–X.7.5) — real-time analysis of direct operator input before it crosses an external trust boundary;
2. **Data-stream supervision** (X.7.7) — observation of flows between components and across declared boundaries against declared expectations and the historical corridor;
3. **Workflow supervision** (X.7.7) — observation of automation and workflow execution for anomalous patterns.

Fraud Watchdog™ remains the specialist beside it: fraud, phishing, and social-engineering prevention on the inbound message surface (Annex IX §IX.3.1 rule 9). Specialist and generalist share the discipline of this Annex — WR Expert™ parameterization where applicable, Enforcement Modes (X.9), the ratchet rule, and finding records (X.10.3).

X.7.2 Duty Area 1 — Operator-Input Egress Analysis

In its egress duty, the WR Guard™ analyzes direct operator input — in particular, prompt input to conversational AI surfaces — already during typing, as locally as the deployment permits, before content crosses an external trust boundary. When active, the user interface marks affected terms, passages, or topic areas in real time and displays the reason for the classification.

Classification SHALL NOT be limited to exact keywords. The Guard also recognizes semantic relationships, indirect statements, combinations of individually innocuous items, and topics that constitute a disclosure risk under the configured WR Expert™ files or the Guard's training.

The operator prompt class. Direct operator input to conversational AI surfaces is an access class of its own, distinct from the component flows of X.5: the actor is the operator, and the Guard's purpose here is protective — preventing the operator's accidental mistakes, not policing an adversary. The response is graded by the assurance class the signed policy assigns to the affected risk and data classes, along a three-tier ladder drawn from the measure set of X.8:

1. **advisory** — real-time marking with the stated reason; the operator proceeds informed;
2. **protective transformation** — the input proceeds only in transformed form: obfuscation, masking, or compute-distribution obfuscation (the Distribution measure of X.8.2; the Distribution Obfuscation tactic of the Canon), applied per the mapping;
3. **full block** — the transmission halts entirely, releasable only per X.8.4 or, for non-releasable classes, not at all.

Every event of every tier — marking, transformation, block, release, override — is logged per X.10.3.

X.7.3 Two Detection Layers — Facts Before Interpretation

Detection is layered, mirroring the facts-before-interpretation discipline of Annex IX (IX.3.1 rules 7–9):

1. **Deterministic Confidentiality Detectors (facts).** Registered material is detected deterministically (X.2). A detector match is a fact, not a finding: the policy-mapped Egress Measure applies unconditionally, in every Enforcement Mode, with no model in the decision path.
2. **Semantic classification (heuristics).** The WR Guard™ model detects what structural matching cannot: paraphrase, indirection, combination, semantic proximity. Guard findings are heuristic in origin; their consequence is governed by the Enforcement Mode (X.9). Detection quality of this layer is an objective in the sense of X.13, never a by-construction property.

X.7.4 Multi-File Consumption and Precedence

A WR Guard™ instance MAY consume several WR Expert™ files simultaneously (organization-wide baseline, device-specific profile, project-specific supplement), each individually versioned and individually admitted. Conflict resolution between files is deterministic and declared in the signed policy — never resolved by the model; the default rule is that the strictest applicable measure prevails, which is ratchet-conforming by construction.

X.7.5 Version Attribution

Every marking, every finding, and every applied measure SHALL be recorded as a Boundary Event Record (X.10.3) carrying the SHA-256 digests of the WR Expert™ file version(s) consulted, the WR Guard™ adapter version, the signed policy version, and the Enforcement Mode in effect for the finding class. It is thereby demonstrable after the fact under which complete rule state a classification and its consequence occurred.

X.7.6 Locality of Analysis

Analysis runs locally within the trusted environment wherever the deployment permits; neither findings nor analyzed content are transmitted off-device for the purpose of this analysis, consistent with the local-evidence discipline of IX.11. Higher verification tiers receive hash commitments only (X.10.4).

X.7.7 Duty Areas 2 and 3 — Data-Stream and Workflow Supervision (Normative)

Beyond operator input, the WR Guard™ supervises the stack's flows and workflows as a heuristic layer over the deterministic record:

1. **Data-stream supervision.** The Guard observes flows between components and across declared boundaries — capability usage, volumes, frequencies, timing, counterparties — against the expectations declared in admitted WR Expert™ files and against the historical

corridor of the local evidence log (the history-based stage of IX.8.7 generalized from proposals to flows). Its inputs are the deployment's own records — Boundary Event Records, capability invocations, deviation events — never raw content beyond what its duty's declared context admits (IX.8.6).

2. **Workflow supervision.** The Guard observes the execution of automations and declared chains — WCR runs and computer-use chains (IX.21.5) — for anomalous sequences, unusual parameter patterns, atypical timing, and out-of-corridor behavior across runs.

The layering discipline of X.7.3 applies unchanged: the deterministic mechanisms remain the authoritative facts — the deviation hard stop (IX.6.5), witnessing (IX.17), detector matches, and the evidence chain itself — and no Guard finding replaces, precedes, or softens any of them. Guard findings in these duty areas are interpretation over the record: under **advisory**, they are surfaced in client chrome and logged; under **enforced**, they trigger the policy-mapped friction — suspending an affected flow or automation pending release, escalating the required Consent Mechanism Class for the next consequential step, or blocking a channel pending administrative review — under the ratchet rule verbatim: a wrong, deceived, or compromised Guard can at most delay legitimate activity, never enable, approve, or execute anything. Every finding is recorded per X.10.3; detection quality in these duty areas is an objective per X.13.2.

X.7.8 Policy Enforcement and Logging within the Guard (Normative)

The WR Guard™ is not confined to producing findings for enforcement elsewhere: the component MAY itself serve as the local enforcement point for the signed policy at its positions — the operator-input path, the supervised streams, the supervised workflows — **enforcing and logging cryptographically signed policy decisions**. This capability resides in the Guard's deterministic enforcement stage, under the following rules:

1. **Verify before enforce.** The enforcement stage SHALL verify, fail-closed and before applying any decision: the signature of the local policy against the administrative authority's key; the policy version against the highest version already accepted (the anti-rollback discipline of IX.4.2 applied to policies — a validly signed stale policy is the rollback attack, not a valid state); and the hash bindings of the WR Expert™ file versions the decision references. A policy that fails any check enforces nothing; the failure itself is sealed as a Boundary Event Record.
2. **Internal separation.** The two stages are separated with a typed interface: the classification stage emits typed, bounded finding records and nothing else; the enforcement stage contains no model and evaluates exclusively the typed finding against the signed risk-class-to-measure mapping. No message type exists through which the classification stage could instruct a release, lower a measure, or alter the mapping — the admission discipline of IX.8.6 applied inside the component. Compromise of the classification stage therefore yields at most false findings, i.e. wrong friction (the ratchet residue); it yields no enforcement authority.
3. **Deterministic execution.** For a given signed policy version, a given WR Expert™ file set, a given typed finding, and a given Enforcement Mode, the enforcement stage's decision is a pure function — reproducible from the sealed record, with no model in the path.
4. **Every decision sealed.** Every enforcement decision — measure applied, escalation raised, suspension imposed, release processed, and equally every refusal to enforce under rule 1 —

is sealed as a Boundary Event Record (X.10.3) binding the signed-policy reference and version, the Expert-file digests, the finding reference, the Enforcement Mode, and the decision taken. The Guard thereby does not merely act on the signed policy; it produces, per decision, the cryptographic proof that it acted on exactly that policy.

Positioning note (informative): where a Guard-enforced measure gates a consequential egress, the Guard's enforcement stage acts as the pre-boundary gate, and the consequence boundary of X.11 remains behind it unchanged — two deterministic checks in series, mutually untrusting in the sense of IX.8.7(4), neither relying on the other having run.

X.8 Graded Egress Measures and Release Paths (Normative)

X.8.1 The Measure Set

The binding treatment of classified content follows the signed egress policy (X.8.3). Depending on its mapping of risk classes to measures, the system can in particular:

1. merely warn the user;
2. block the transmission until an explicit release;
3. mask individual terms or passages;
4. pseudonymize information;
5. obfuscate sensitive content;
6. distribute information across several separated processing paths (**Distribution**, X.8.2);
7. release only abstracted or summarized content;
8. process certain content exclusively locally;
9. encrypt data for an explicitly authorized recipient; or
10. reject the entire request.

X.8.2 Distribution

Distribution designates the controlled splitting of information across several models, services, or processing steps such that no single external recipient receives the complete sensitive context. Each partial recipient's share is covered by that recipient's own egress capability under its own handshake (X.3); recombination outside the trust domain is not an authorized operation, and each partial crossing is evidenced individually (X.10.2).

X.8.3 Measures Are Declared Paths

Each measure is a pre-declared, policy-mapped processing path. The signed egress policy — authored by the administrative authority, cryptographically signed, versioned — maps the risk classes of the admitted WR Expert™ files onto measures, per class and per target channel. The WR Guard™'s role with respect to measures is exactly the router discipline of IX.16.4: it may propose or select the applicable protective path within the declared mapping; it never creates a path, never widens one, and never selects a path less protective than the mapping mandates. The binding decision — release, transformation, encryption, local processing, or blocking — is made by the deterministic policy and capability layer.

X.8.4 Release Paths

Where a measure blocks pending release, the release path is itself policy-defined, using the Consent Mechanism Classes of IX.18 unchanged:

1. Policy MAY declare a risk class **non-releasable** at the operator level — release only by the administrative authority, or not at all.
2. Policy MAY require, per risk class, a minimum Consent Mechanism Class for release (e.g. C2/Passkey instead of C1/tap), with the meet-or-exceed rule of IX.18.3 applying unchanged, and MAY additionally require a stated justification, carried in the record.
3. Every release is an interactive consent event: it is PoAE™-recorded with the Intent Hash (IX.19.2) computed over the presentation **as marked** — the operator demonstrably confirmed in view of the classification and its stated reason. A release record whose Intent Hash does not resolve to the marked presentation is an invalid consent record (IX.3.4) and a deviation (IX.6.5).

No finding, no absence of a finding, and no model output pre-satisfies any release (Annex II §II.2.5): an unclassified input is not thereby an authorized one — the egress capability covering the target channel remains the necessary condition for every crossing, with or without a Guard in the path.

X.9 Enforcement Modes for Local Analysis Components (Normative)

The deployment's local analysis components — at this version: Fraud **Watchdog**™ (Annex IX §IX.3.1 rule 9; the fraud-prevention specialist on the inbound message surface) and **WR Guard**™ (X.7; the general supervisor of data streams and workflows) — operate under a policy-declared **Enforcement Mode**, declared in the signed local policy **per component and per finding class**:

- **advisory** — findings are surfaced at the relevant decision points, visually distinguished as analysis findings (never as verified facts), and do not bind the flow;
- **enforced** — findings trigger the policy-mapped measure bindingly: blocking, masking, quarantining message actions, escalating the required Consent Mechanism Class, or any other friction-adding measure of the applicable mapping. An enforced measure is neither softenable nor dismissible by the model that produced the finding.

Per-finding-class declaration is deliberate. A deployment MAY, for example, run Fraud Watchdog™ **advisory** for stylistic suspicion findings while enforcing its brand-impersonation cross-check on unauthenticated channels (blocking link opening pending release), and MAY run WR Guard™ **advisory** for low risk classes (real-time marking only) while enforcing registered-confidential classes unconditionally.

The following invariants hold in both modes and are not policy-variable:

1. **Epistemic status is mode-independent.** A heuristic finding remains heuristic in enforced mode; the mode changes consequence, never confidence. Detection quality remains an objective (X.13) in both modes.
2. **The ratchet discipline is mode-independent.** In either mode, findings only ever add friction (IX.8.7): they SHALL NOT suppress, soften, or replace any deterministic verdict or

- warning (IX.3.1 rule 8 unchanged), SHALL NOT upgrade trust, SHALL NOT reduce any required consent tier or Consent Mechanism Class, and SHALL NOT pre-satisfy or substitute for any consent. **enforced** is maximal friction, not new authority.
3. **Facts have no mode.** Deterministic verdicts and detectors — the Channel Provenance Record (IX.3.1 rules 7–8) and the Deterministic Confidentiality Detectors (X.7.3) — are not findings and know no advisory setting; their consequences apply unconditionally per their governing rules.
 4. **Mode changes are policy events.** The Enforcement Mode is set and changed exclusively through the signed local policy — never at runtime, never by the component, never by message, publisher, or counterparty content. A counterparty SHALL NOT be able to lower any mode or mapped measure below what the operator's policy demands (IX.18.3 applied to modes); a counterparty MAY declare stricter minimums for its own surfaces.
 5. **The effective mode is evidence.** Every recorded finding carries the Enforcement Mode in effect for its class at the time (X.7.5, X.10.3).

Defaults, absent a policy declaration: Fraud Watchdog™ operates **advisory** — its Annex IX characterization, which this Annex thereby preserves unchanged as the default mode, Secure Browse escalation per IX.3.1 rule 8 and IX.9 remaining available; WR Guard™ risk classes declared in an admitted confidentiality-profile WR Expert™ file operate **enforced**. Nothing in this section modifies any rule of Annex IX; it generalizes an existing behavior into a declared parameter with the existing behavior as its default.

X.10 The Evidence Discipline — Every Crossing, Every Finding (Normative)

X.10.1 Principle

The trust boundary to external services — and every declared trust-domain boundary of the deployment — is not merely monitored: it is **cryptographically evidenced**. Monitoring observes; evidence proves. Every crossing of a declared boundary and every finding of a local analysis component SHALL produce a Boundary Event Record sealed into the deployment's append-only, hash-chained local evidence log under the Execution Receipt discipline of IX.19 (monotonic per-domain sequence number, previous-record digest), so that removal, reordering, or insertion of records is cryptographically detectable at Tier L without any external party (claims discipline per IX.4.4(4): detectable, not "tamper-proof").

X.10.2 Crossing Records

A Boundary Event Record for a crossing SHALL bind, at minimum: the governing reference — Handshake Contract (Public or Internal) or External Service Admission — and its version; the direction (ingress/egress) and the invoked directional capability; for External Invocations, the operation class and the satisfied Invocation Assurance Class; the data classes crossing (or their commitment, per data-class policy — content itself is not required to enter the record); applied Egress Measures with their parameters' commitments (e.g. the masking map's digest, the Distribution split identifiers); the counterparty and channel; the consent or policy-decision reference where the crossing required one, including the Intent Hash for interactive releases (X.8.4);

timestamp, sequence number, and chain digest per IX.19. Where the crossing is a consequential effect under IX.8.4, the Boundary Event Record and the Execution Receipt are one record, not two.

X.10.3 Finding Records

Every finding of a local analysis component — WR Guard™ classification, Fraud Watchdog™ assessment — SHALL be recorded with: the component and adapter version; the consulted WR Expert™ file digests and policy version (X.7.5); the finding class and stated reason; the Enforcement Mode in effect and the measure applied or proposed; the operator's response where one occurred (release, override, abandonment), with its Consent Mechanism Class and Intent Hash where interactive; timestamp, sequence number, and chain digest. Findings are evidence regardless of consequence: an **advisory** finding the operator ignored is as provable as an **enforced** block — which is what makes the record usable for audit, for the meta-feedback dampening discipline (IX.8.7(5)), and for demonstrating due diligence after an incident.

X.10.4 Verification Tiers and Privacy

Boundary Event Records participate in the Verification Tiers of IX.20 unchanged: always verifiable at **Tier L** (local hash chain); optionally committed to the **Tier C** countersigned transparency log and anchored at **Tier X** — with the tier invariants of IX.20 in force verbatim: validity is established at seal; higher tiers add independent verifiability, never validity and never authority; tier selection is local policy plus operator choice, and no counterparty can force a lower tier; and **no tier transmits operator content off-device** — Tiers C and X operate exclusively on digests, so the evidence discipline itself opens no disclosure channel. What the deployment gains is the ability to prove, to itself and where it chooses to third parties, *that* and *under which authority* every crossing occurred — without the proof infrastructure ever holding what crossed.

X.11 Position in the Consequence Boundary (Normative)

Egress introduces no new boundary. A transmission across a trust boundary is an externally observable consequence (IX.8.4(3)); it exists only through a declared enforcement point, under a valid Execution Capability covering channel, recipient, data classes, and purpose (X.3, X.5), and it is receipted and verifiable under the ordinary machinery (IX.17, IX.19, IX.20; X.10). In its egress duty, the WR Guard™ is positioned **before** that boundary as a detection and classification stage — architecturally the position of Plausibility Verification (IX.8.7), sharing its ratchet discipline — and the Egress Measures of X.8 are transformations applied on the path **to** the boundary, never bypasses of it.

The division of authority is therefore exact, and closes this Annex where the corpus always closes: the AI serves semantic recognition, classification, and the selection of suitable protective measures; the authority to release or execute arises exclusively from the valid, cryptographically verifiable policy and capability chain — signed policies, minted capabilities, deterministic enforcement, sealed proof.

X.12 Enforcement Substrate and Operating-System Integration (Informative)

The enforcement architecture MAY initially operate at application level: ingress and egress mediated through the client core and its controlled interfaces — the deployment form of this version. Stronger enforcement depth is a staged direction, extending the substrate mapping of Annex IX §IX.22: every inbound and outbound operation MAY carry a verifiable execution token — the Execution Capability of IX.8.5, or an attenuated derivation of it — such that the operating system's security infrastructure participates directly in capability-based information flow: LSM-based mandatory access control frameworks (SELinux, AppArmor), **extended rather than replaced**, verifying capability tokens before permitting process communication, file access, inter-process messaging, or network transmission; measured-integrity and kernel-observation mechanisms corroborating the pre-/post-state witnessing; hardware trust anchors holding the signing keys; and Boundary Event Records (X.10) sealed for the permitted operations. The trust boundary thereby becomes independent of process ownership, execution location, and network topology — the operational form of X.4. Neither stage changes any normative rule: the token is the existing capability, the record the existing receipt; what deepens is the enforcement substrate, not the trust architecture. Per X.0.1, the achievable depth per platform, and the correctness of any OS-level integration, are objectives (X.13), not claims.

X.13 Security Model: Properties, Objectives, Assumptions, Non-Goals

X.13.1 Properties by Construction (Normative claims)

- Locality confers no authority: internal components, sandboxes, field devices, and local models hold exactly their handshake-negotiated capabilities; a sandbox inherits no host authority; the local model holds no capability material (X.4).
- Every data relationship is governed under exactly one of the three governance classes — Public Handshake, Internal Handshake, or External Service Admission — with per-direction declarations; an undeclared direction is an empty set, a non-admitted external service is unreachable as an authorized target, and a flow within an undeclared direction is a deviation (X.3).
- Every External Invocation is authorized individually at its declared Invocation Assurance Class, with mandated transformations as preconditions where declared, and no service can lower any assurance requirement of the operator's policy (X.3.3).
- No non-resident access to the WR Graph™ — field devices, dedicated sandboxes, external sources — is representable as an authorized operation without a per-operation cryptographic capability declaring scope, operation, recipient, purpose, protections, validity, and revocation; the requirement attaches to the requester class, not to content marking. Resident access by local models conveys information only and never disclosure authority, and every access of every class is logged (X.5).
- Deterministic Confidentiality Detector matches apply their policy-mapped measure unconditionally, in every Enforcement Mode, with no model in the decision path (X.7.3).
- No finding, absence of a finding, or model output of any local analysis component, in any Enforcement Mode, can release data, soften a deterministic verdict, reduce a consent

requirement, or authorize an egress; the ratchet discipline is mode-independent, and mode changes are exclusively policy events a counterparty cannot lower (X.8, X.9).

- Every enforcement decision of the WR Guard™'s deterministic stage is executed only against a signature- and version-verified policy with hash-verified WR Expert™ bindings, is a reproducible pure function of policy, files, finding, and mode, and is sealed into the evidence chain — including refusals to enforce, which fail closed (X.7.8).
- Every release of a blocked egress is an interactive consent event bound by Intent Hash to the marked presentation the operator actually confirmed; a record that does not resolve to that presentation is invalid (X.8.4).
- Every crossing of a declared boundary and every analysis finding is sealed into the append-only, hash-chained evidence log and verifiable at the configured tiers, with content never leaving the device through the evidence path (X.10).
- Under Distribution, no single external recipient's capability covers the complete sensitive context, and recombination outside the trust domain is not an authorized operation (X.8.2).

X.13.2 Objectives Pending Validation (Informative)

The semantic detection quality of the WR Guard™ — in particular its false-negative discipline against paraphrase, indirection, and combination; the completeness and maintainability of deployment-authored WR Expert™ files; real-time classification robustness under typing-latency constraints; the anomaly-detection quality and false-positive discipline of the Guard's data-stream and workflow supervision (X.7.7); the operational safety of the WR Guard™ as a model-based stage under the ratchet rule (alongside the corresponding Fraud Watchdog™ objective of IX.12.2); the effectiveness of Distribution against recombination by colluding recipients; the completeness of boundary declaration in real deployments (an undeclared boundary produces no records); and the depth and correctness of any OS-level enforcement integration (X.12). These require implementation, testing, adversarial analysis, and audit before being claimed.

X.13.3 Trust Assumptions (Informative)

The deployment's administrative authority as author of WR Expert™ files and the signed egress policy — a negligent or malicious policy author is outside the technical trust model, mitigated only by versioned evidence and auditability; the correct implementation of the WR Guard™'s deterministic enforcement stage (a deterministic component in the same sense as the Finalizer trust assumption of IX.12.3); platform integrity of the participating devices (hardware keystore, process isolation); the Annex VII attestation machinery for handshake identities; and, for Tiers C and X, the assumptions of IX.12.3 unchanged — noting that per IX.20 neither tier is validity-conferring.

X.13.4 The Honest Boundary (Normative)

Within an **already-authorized free-text channel** — an operator typing into a cloud prompt for which an egress capability legitimately exists — the two detection layers of X.7.3 are risk reduction, not a by-construction guarantee. Content that is neither registered (layer 1) nor semantically recognized (layer 2) crosses within the authorized scope; what remains by construction even then is that the crossing itself is capability-bounded to the channel's declared data classes and evidenced (X.10). This Annex accordingly avoids, and derived material SHALL avoid, absolute statements such as "data can never leak" or "the system guarantees complete security." The accurate formulations are: *the architecture is designed to conduct protected information flows exclusively*

over explicitly authorized and cryptographically verifiable capability paths, and: within the controlled rendering and execution pipeline, data transfers without a valid capability are not accepted as authorized system operations. The more completely an organization registers and marks its confidential holdings (X.6, X.7.3), the more of the protection migrates from objective to construction.

X.13.5 Non-Goals (Normative)

The system bounds, transforms, and evidences technical information flows. It does not and cannot guarantee that an authorized recipient handles received data well, that a colluding set of Distribution recipients cannot attempt recombination from their shares, that policy authors declare the right scopes, or that evidence is acted upon. The architecture reduces the technical disclosure surface and makes crossings provable; it replaces neither organizational governance nor the assessment of counterparties.

X.14 Relation to Prior Work and Claimed Contribution (Informative)

The constituent mechanisms of this Annex are established practice and are not claimed as novel. The closest neighbours:

- **Zero-trust architectures** (BeyondCorp-class designs; NIST SP 800-207) establish "never trust by network location; authenticate every request." They authenticate identities and authorize requests at the service perimeter; they do not bind per-direction data-class envelopes to a consent record, govern semantic content of authorized flows, or produce per-crossing cryptographic receipts.
- **Data-loss-prevention suites and CASB gateways** (enterprise DLP, cloud access security brokers) establish content inspection at egress points, including for AI-prompt surfaces. Their detection is a detached filter: rules live in the gateway rather than as governed, hash-pinned artifacts; findings bind no consent machinery; logs are administrative, not hash-chained evidence; and the inspecting component is trusted rather than mechanically stripped of release authority.
- **Capability-based security** (object capabilities; Capsicum; seL4-class systems) establishes unforgeable, attenuated authority tokens. They govern program authority within a system; they do not carry negotiated, per-direction data-class scopes across organizational trust boundaries under consent, nor bind human authorization evidence into the token's use.
- **Decentralized information flow control** (Flume/HiStar-class labels) establishes tracking and confinement of data flows by label. Labels confine propagation; they are neither handshake-negotiated contracts nor consent-bound, and produce no receipts.
- **Mandatory access control frameworks** (SELinux, AppArmor) establish policy-enforced confinement — the enforcement substrate this Annex explicitly builds upon (X.12) rather than claims.
- **PII redaction and prompt-firewall tooling** for LLM applications establishes masking and filtering of model inputs. These are point mitigations: heuristic, unattributed to versioned governed knowledge, without graded consent-bound release paths, and without cryptographic proof of what was transformed and what crossed.

- **Audit logging and SIEM pipelines** establish comprehensive event capture. They are observational and administratively mutable in trust terms; they are not append-only hash chains bound to consent records and capabilities, verifiable without trusting the log operator.

Each neighbour supplies a mechanism; none supplies the contract. The claimed contribution of this Annex is the normative composition under the Canon's governance model: locality-independent trust in which internal and external components are governed identically; one governance fabric with three classes — Public Handshake, Internal Handshake, and External Service Admission — under which every data relationship declares per-direction capability envelopes, so that consent records stay permanently accurate while content moves, with non-WR services honestly governed as unilateral admissions authorized per invocation at declared Invocation Assurance Classes rather than misdescribed as handshake parties; graph access as a capability-scoped disclosure path with purpose, recipient, protection, and revocation bound into the authorization; expert knowledge as a governed, hash-pinned artifact class (WR Expert™) shared with the system's entire model-specialization machinery rather than a filter's rule list; a general local supervision component (WR Guard™) over the stack's data streams and workflows, layered strictly above deterministic detection and mechanically incapable of authorizing in any Enforcement Mode; graded, policy-mapped egress transformations — through masking, obfuscation, recipient-bound encryption, and Distribution — as declared paths selected under the router discipline, mandatable as invocation preconditions; release only through graded, Intent-Hash-bound consent over the marked presentation; and every crossing and every finding sealed into an append-only, tier-verifiable evidence chain that carries digests, never content. Individually, these ideas have neighbours; to the author's knowledge, following review of the adjacent work above, no existing system composes them in this governed, consent-bound, evidence-producing form.

X.15 Open Specification Items (Informative)

The WR Expert™ file class specification (format, section grammar, semantic-scope declaration language, signing envelope) and its PoAC™ admission profile; the Internal Handshake profile declarations within the Annex VII machinery (component identity classes, per-direction capability-set schema); the External Service Admission schema (endpoints and identity anchors, operation classes, per-direction data-class ceilings, mandatory-transformation declarations) and the Invocation Assurance Class taxonomy with its policy declaration format; the capability field grammar for graph and data access (X.5); the egress-policy schema (risk-class-to-measure mapping, per-channel declarations); transformation profiles for masking, pseudonymization, obfuscation, and abstraction; the Distribution scheme (split declaration, per-recipient capability binding, recombination-prohibition semantics, share identifiers); the Enforcement Mode declaration schema (component, finding class, mode, mapped measures, release minimums); the Boundary Event Record serialization for crossings and findings, including commitment formats for data classes and transformation parameters; boundary-declaration completeness criteria per deployment class; the release-record justification carriage; and the WR Guard™ evaluation methodology (detection-quality benchmarks, false-negative discipline, latency envelope).

X.16 Application Examples (Informative)

1. **The accidental cloud prompt.** An employee types company-internal figures into a cloud AI chat. A Deterministic Confidentiality Detector matches a registered project identifier while

- typing; the passage is marked with the stated reason. The class is **enforced**, mapped to *block pending release* with minimum class C2: on send, transmission halts before the trust boundary; the client offers the masked variant or local-only processing; release of the original requires a Passkey confirmation over the marked view. Either the data never left — with the finding itself sealed as evidence — or a sealed record proves who released it, when, in view of which warning, under which Expert-file, adapter, and policy versions.
2. **The field device.** A field device requests context from the host WR Graph™. Its Internal Handshake's egress set (host → device) covers one declared graph branch, read-only, for a declared purpose and duration; its ingress set (device → host) covers telemetry of declared classes. A request outside the branch is unrepresentable as an authorized operation; the attempt is a deviation with a Boundary Event Record. Neither direction implies the other.
 3. **The sandbox.** A depackaging sandbox processes a received artifact. It holds no host data, keys, or graph access; the validated result re-crosses into the client core under the declared ingress capability of its Internal Handshake, the crossing sealed as evidence — and nothing else crosses at all.
 4. **The locally produced Office file.** A document is generated locally from data passed in under explicit capabilities. Every ingredient is traceable to a capability-covered, evidenced transfer; the file's subsequent egress toward any recipient is itself a crossing under X.10, with its applied transformations committed in the record.
 5. **The distributed model query.** A task requires external model capacity over sensitive context. Under Distribution, the context is split across two providers such that neither receives the complete sensitive picture; each share travels under its recipient's own External Service Admission and egress ceiling, each crossing is evidenced individually, and the recombination exists only locally.
 6. **The external RAG query.** A workflow needs an external retrieval. The search endpoint is admitted with operation class *retrieval*; its Invocation Assurance Class permits C0 under an admitted policy rule for abstracted queries only. The concrete query is Guard-checked and — containing a customer name — mandatorily pseudonymized per the admission before egress; the query itself is treated as disclosure. The returned passages enter as a declared free-text source under the context-admission discipline (IX.8.6) — consumable as information, incapable of instruction — and the invocation's Boundary Event Record binds admission version, operation class, satisfied assurance class, applied transformation, and both directions' data-class commitments.

§X.17 Version History

v1.0 — 2026-07-18 — Initial normative specification: locality-independent trust as generating principle (membership grounds no access; sandboxes inherit nothing; the local model as an untrusted probabilistic component under the router discipline); the governance fabric with three classes — Public Handshake (Annex IX), Internal Handshake (registered designation; Annex VII machinery; formation as policy event), and External Service Admission (registered designation; strictly not a handshake — the non-WR counterparty forms and attests nothing; unilateral, deployment-side, signed admission with per-invocation authorization graded by Invocation Assurance Classes, mandated transformations as invocation preconditions for higher data classes, the query itself treated as disclosure on egress and returned free text bounded by the context-admission discipline on ingress, and a consent-gated upgrade path to a handshake class should a

provider later implement WR) — under which every data relationship of the system is governed, with per-direction capability sets (ingress/egress declared separately; undeclared direction = empty set; the Scope Envelope discipline applied to the interior; directional grammar uniform across all three classes); capability-scoped WR Graph™ and protected-data access, structure-agnostic (regions/nodes/branches/classes, operation, recipient, purpose, protective measures, validity, revocation; graph internals deliberately unfixed) and graded by requester class — resident access for local models as working context conveying information and never disclosure authority, per-operation cryptographic approval always required for non-resident internal device classes (field devices, dedicated sandboxes) and for external sources regardless of content marking, every access of every class logged; the operator prompt surface as its own protective access class with the assurance-graded three-tier response ladder — advisory marking, protective transformation (obfuscation, masking, compute-distribution obfuscation), full block — every event logged; the WR Expert™ file established as the system's governed skill-file artifact class under PoAC™ discipline with the confidentiality profile as one instantiation; the WR Guard™ established as the general local supervision component over the stack's data streams and workflows — Fraud Watchdog™ remaining the inbound fraud-prevention specialist beside it — with three duty areas: real-time operator-input egress analysis (semantic analysis during typing before the external trust boundary, real-time marking with stated reasons, multi-file consumption with deterministic strictest-wins precedence, full version attribution), data-stream supervision, and workflow supervision against declared expectations and the historical corridor, and with a two-stage internal architecture in which a deterministic enforcement stage verifies (signature, anti-rollback version, Expert-file hash bindings; fail-closed), executes, and seals cryptographically signed policy decisions as reproducible pure functions, the probabilistic classification stage confined to typed findings with no message type toward release; two-layer detection under facts-before-interpretation (Deterministic Confidentiality Detectors applying mapped measures unconditionally in every mode; semantic classification as heuristics with detection quality a pending objective); ten graded egress measures as declared, policy-mapped paths (warn, block-pending-release, mask, pseudonymize, obfuscate, Distribution with per-recipient capability coverage and no-recombination semantics, abstraction, local-only, recipient-bound encryption, reject), selected under the router discipline, with release paths using the C0–C4 machinery unchanged (per-class minimums, non-releasable classes, Intent Hash over the marked presentation, PoAE™-recorded); Enforcement Modes advisory/enforced per component and per finding class for the local analysis components (Fraud Watchdog™ inbound, WR Guard™ outbound), with mode-independent invariants (epistemic status, ratchet discipline, facts have no mode, mode changes as policy events with counterparty meet-or-exceed, effective mode in evidence) and prior behaviors preserved as defaults, Annex IX unmodified; the evidence discipline — every boundary crossing and every analysis finding sealed as a Boundary Event Record into the append-only hash chain (crossing records binding contract, direction, capability, data-class commitments, transformation commitments, consent references; finding records binding component, versions, mode, measure, and operator response), verifiable at Tiers L/C/X with digests only ever leaving the device; position in the existing consequence boundary (transmission as consequence per IX.8.4(3); the Guard in the Plausibility-Verification position; no new enforcement layer); staged enforcement substrate (application-level first; capability-token verification carried into MAC frameworks as extension, never replacement); security model with by-construction properties, pending objectives, trust assumptions, the honest boundary of the authorized free-text channel stated normatively, and non-goals; relation to prior work (zero trust, DLP/CASB, capability systems, DIFC, MAC, prompt-

firewall tooling, audit pipelines) with the claimed contribution as governed composition; open specification items; five application examples.

v1.0 — 2026-07-28 WR Guard adapter section: first-party issuance and signing by Optirando, hash- and signature-gated hot-swap admission, per-class minimum-version (rollback) policy, adapter-version-bound verdicts, and governed idle-phase advance analysis of depackaged inbound content.

v1.1 — 2026-08-07: Refined WR Guard™ architecture: WR Guard™ clarified as the security supervision and routing framework; Fraud Watchdog™, Ingress Watchdog™, and Process Watchdog™ defined as specialized LoRA-class security adapters with governed mode selection and contextual hot-swapping.

License Notice. This Annex forms an integral part of the Optirando™ specification and is licensed under the same terms as BEAP™, WR Desk™, and PoAE™.